Zero Knowledge Proofs for Logistic Regression Predictions and Accuracy

A Project Report Submitted

in Partial Fulfilment of the Requirements

for the Degree of

Bachelor of Technology

in

Computer Science and Engineering

with Specialization in

Cyber Security

by

Santhosh Cheemala - 2022BCY0009

Shravasti Ohol - 2022BCY0012

Buddala Kamalakar - 2022BCY0050

to

DEPARTMENT OF CSE-CYBER SECURITY

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

KOTTAYAM-686635, INDIA

November 2025

DECLARATION

We, **Santhosh Cheemala** (2022BCY0009), **Shravasti Ohol** (2022BCY0012), and **Buddala Kamalakar** (2022BCY0050), hereby declare that this report entitled **"Zero Knowledge Proofs for Logistic Regression Predictions and Accuracy"** submitted to IIIT Kottayam is an original work carried out under the supervision of **Dr. S. Jai Ganesh**. It has not been submitted elsewhere for any degree or diploma.

Santhosh Cheemala

Kottayam-686635

Shravasti Ohol

November 2025

Buddala Kamalakar

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Zero Knowledge Proofs for Logistic Regression Predictions and Accuracy"** submitted by **Santhosh Cheemala**, **Shravasti Ohol**, and **Buddala Kamalakar** to IIIT Kottayam towards partial fulfillment of the B.Tech degree has been carried out under my supervision. It has not been submitted elsewhere for the award of any degree.

Kottayam-686635 **Dr. S. Jai Ganesh**

November 2025 Project Supervisor

ABSTRACT

The growing reliance on machine learning models in sensitive domains such as healthcare, finance, and cyber security has increased the need for trustworthy and privacy-preserving prediction systems. Traditional verification methods expose confidential model parameters and data, raising critical concerns about security and intellectual property. To address these challenges, this work proposes a Zero-Knowledge Proof (ZKP) framework for verifying logistic regression predictions and model accuracy without revealing sensitive information.

Our approach leverages PLONK-SNARK, a modern ZKP protocol, to construct arithmetic circuits representing logistic regression computations. The framework securely encodes the model's weights and bias as secret inputs, enabling a verifier to confirm the correctness of predictions and accuracy metrics without accessing the underlying data. Non-linear functions such as the sigmoid are efficiently handled through lookup tables, significantly reducing computational overhead while maintaining cryptographic soundness.

Our system ensures data confidentiality, model privacy, and verifiable inference in a decentralized setting. The implementation, developed using the GNARK library in Go, supports batch verification for scalable performance across datasets.

This framework bridges the gap between machine learning transparency and cryptographic privacy, offering a robust foundation for secure model verification in privacy-critical applications—ultimately fostering trust, compliance, and accountability in AI-driven decision systems.

Contents

List of Figures ix

List of Tables xi

1 Introduction 1

1.1 Background 2

1.2 Motivation 2

1.3 Problem Statement 3

1.4 Project Scope 3

2 Literature Review 5

2.1 Introduction 5

2.2 Scalable Zero-Knowledge Proofs for Non-Linear Functions in Machine Learning 6

2.3 Mystique: Efficient Conversions for Zero-Knowledge Proofs 6

2.4 Zero-Knowledge Proofs for Decision Tree Predictions and Accuracy . 7

2.5 Proof of Training for Logistic Regression 7

2.6 Efficient Zero-Knowledge Proofs for Deep Learning Training 8

2.7 Benchmarking Non-Interactive Zero-Knowledge Proof Protocols . . . 8

2.8 ZKP-Based Verifiable Machine Learning Survey 9
2.9 Research Gaps Identified 9
2.10 Conclusion 10

3 Theoretical Background **11**
3.1 Zero-Knowledge Proofs 11
3.2 Arithmetic Circuits and Constraint Systems 12
3.3 Polynomial Commitment Schemes 12
3.4 Sigmoid Function and Approximation 13
3.5 Logistic Regression Model 14
3.6 System Architecture 15
3.7 Proof Generation Workflow 16

4 Detailed Implementation **17**
4.1 System Architecture 17
4.1.1 Overview 17
4.1.2 Technology Stack 18
4.2 Dataset Handling 18
4.2.1 Dataset Overview 18
4.2.2 Dataset Generation and Training 19
4.3 Model Design 19
4.3.1 Logistic Regression Formulation 19
4.3.2 Fixed-Point Encoding 19
4.4 Circuit Construction 20
4.4.1 Circuit Layers 20
4.4.2 Constraint Count 20

4.5 Proof Generation Workflow 21
4.5.1 Setup Phase 21
4.5.2 Individual Proof Generation 21
4.5.3 Batched Proof Implementation 22
4.6 Performance Optimizations 23
4.6.1 Lookup Table Reduction 23
4.6.2 Circuit Caching and Precision Control 24
4.7 Security Considerations 24
4.8 Summary 24

5 Results and Observations **25**
5.1 Proof Size Analysis 25
5.2 Batched vs Individual Performance 26
5.3 Throughput Analysis 27
5.4 Scalability Characteristics 28
5.5 Batch Processing Consistency 29
5.6 Cumulative Optimization Impact 30
5.7 Security Validation 31
5.7.1 Test 1: Valid Proof Acceptance 31
5.7.2 Test 2: Tampered Proof Detection 32
5.7.3 Test 3: Witness Tampering Detection 32
5.8 Limitations and Observations 33
5.8.1 Sequential Processing Constraint 33
5.8.2 Setup Time Overhead 34
5.8.3 Batch Size Trade-off 34
5.8.4 Positive Observations 34

5.9 Summary 34

6 Conclusion and Future Work **36**

6.1 Conclusion 36

6.1.1 Insights Gained 37

6.2 Future Work 37

6.3 Final Remarks 38

Bibliography **40**

List of Figures

3.1 ZKLR system architecture showing modular component design 15

3.2 End-to-end proof generation and verification workflow 16

4.1 Setup time remains constant across different dataset sizes 21

4.2 Proof generation and verification for individual samples 22

4.3 Proof generation and verification for batched samples 23

5.1 Proof size remains constant at 584 bytes for both individual and batched circuits, demonstrating PLONK's succinctness property . . . 26

5.2 Performance comparison showing batched approach achieves 1.6× speedup by reducing proof count from 3,000 to 150 26

5.3 Batched approach achieves 2.5 samples per second throughput 28

5.4 Linear scalability: proving time scales linearly with number of batches while proof size remains constant 29

5.5 Batch processing times remain consistent with minimal variance across all 150 batches 29

5.6 Test 1 terminal output showing successful verification of valid proof . 31

5.7 Test 2 terminal output demonstrating detection of tampered proof bytes 32

5.8 Test 3 terminal output showing rejection of proof with tampered witness values 33

List of Tables

5.1 Cumulative impact of MaxInput optimization and batching strategy . 30

Chapter 1

Introduction

Modern machine learning systems are increasingly embedded in sensitive decision-making pipelines such as healthcare diagnostics, fraud detection, cybersecurity monitoring, financial risk assessment, and identity verification. As these models grow in influence, the need for verifiable and trustworthy outputs becomes critical. In many practical deployments, a client interacts with a remote model via an API or cloud service without access to the underlying parameters or computation process. This lack of transparency raises concerns regarding trust, fairness, and correctness, especially when decisions directly impact user safety, financial transactions, or regulatory compliance [10].

Zero-Knowledge Proofs (ZKPs) offer a cryptographic solution to these challenges by enabling verifiable computations without revealing internal model details. This thesis explores how ZKPs can be applied to logistic regression to enable privacy-preserving verifiable predictions and accuracy reporting [8].

1.1 Background

Zero-Knowledge Proofs, formalized by Goldwasser, Micali, and Rackoff [7], are cryptographic protocols that allow a prover to convince a verifier that a statement is correct without revealing any additional information beyond its correctness. A ZKP system satisfies three essential properties:

Completeness: an honest prover always convinces the verifier; **Soundness**: a dishonest prover cannot produce a valid proof for a false statement except with negligible probability; **Zero-knowledge**: the proof leaks no information about secret values or intermediate computations.

These properties make ZKPs ideal for machine learning environments where revealing the model structure, parameters, or training data would be undesirable or unsafe. In particular, ZKPs enable verifiable inference and evaluation without compromising confidentiality [5].

1.2 Motivation

Machine learning models are often viewed as valuable intellectual property. Providers must protect model weights from extraction attacks while ensuring that clients can trust the predictions. Meanwhile, end-users increasingly demand transparency and verifiability, especially in domains where decisions require justification or compliance guarantees [15].

Traditional approaches to verification may require sharing internal model parameters, openly exposing datasets, or depending on a trusted auditor—all of which compromise privacy. ZKPs remove the need for trust by enabling the prover to demonstrate correct computation while keeping all sensitive information hidden [4].

Thus, the motivation behind this work is twofold: (1) to allow clients to validate predictions without accessing the model internals, and (2) to preserve the confidentiality of model weights, inputs, and intermediate results while maintaining provable correctness.

1.3 Problem Statement

Existing verification mechanisms for machine learning models suffer from limitations such as revealing sensitive parameters, dataset exposure, or dependency on trusted third parties. These limitations pose significant risks, including model extraction, information leakage, and unfair evaluation [**?**].

Therefore, the core problem addressed in this thesis is:

> *How can Zero-Knowledge Proofs be used to design an efficient privacy-preserving framework for logistic regression that allows verifiable predictions and accuracy validation without revealing model parameters or sensitive data?*

1.4 Project Scope

This thesis focuses on the development of a practical zero-knowledge framework for logistic regression. The work begins with designing arithmetic circuits that efficiently capture the computation of logistic regression while remaining compatible with the constraints of a PLONK-based proof system [6]. Non-linear components, particularly the sigmoid activation, are implemented using an optimized lookup-table mechanism to ensure both accuracy and computational feasibility [8]. Zero-knowledge proofs

are generated using GNARK's PLONK backend, which provides an efficient proving system supported by modern cryptographic primitives [2].

The system is evaluated in terms of proving time, verification time, and overall proof size, enabling a detailed performance assessment. Finally, the implementation demonstrates how correct predictions and accuracy reports can be verified securely in a lightweight and privacy-preserving manner, making it suitable for real-world deployment scenarios.

These components collectively contribute toward building a privacy-preserving verifiable inference pipeline for classical machine learning models. The results provide valuable insights for extending ZKP techniques to more complex models and scalable real-world deployments [13].

Chapter 2

Literature Review

2.1 Introduction

The increasing adoption of machine learning models in sensitive domains such as finance, healthcare, and authentication has raised concerns regarding their verifiability, privacy, and correctness. Zero-Knowledge Proofs (ZKPs) have emerged as a powerful cryptographic tool to validate computations without revealing sensitive model parameters or data. This chapter presents a structured review of recent literature on verifiable machine learning, efficient ZKP techniques, and ZKP-based optimization strategies, focusing on contributions relevant to building verifiable logistic regression models using ZKPs.

2.2 Scalable Zero-Knowledge Proofs for Non-Linear Functions in Machine Learning

Hao et al. (2025) introduced a scalable ZKP framework for handling non-linear functions in machine learning, including sigmoid, ReLU, normalization, and other activation functions [8]. Their framework employs digit decomposition and lookup-table-based computation to achieve 50–179× runtime improvement compared to previous ZKP systems.

Insights: This work underscores the effectiveness of lookup-table-based techniques for efficient non-linear function evaluation, directly influencing our approach to implementing sigmoid computation in logistic regression circuits.

Limitations: The system primarily targets deep learning models, with limited optimizations specific to logistic regression.

2.3 Mystique: Efficient Conversions for Zero-Knowledge Proofs

Weng et al. (2021) developed Mystique, a ZKP framework enabling efficient conversions between arithmetic and Boolean circuits to support large-scale machine learning inference [14]. The system enhances interoperability and performance for matrix-based operations in ZKP environments.

Insights: Mystique demonstrates the benefits of modular circuit construction and efficient arithmetic–Boolean translation, which inspire the layered design of our ZKLR circuit.

Limitations: The system is optimized for complex neural networks, making it

heavy for lightweight models such as logistic regression.

2.4 Zero-Knowledge Proofs for Decision Tree Predictions and Accuracy

Zhang et al. (2020) proposed a ZKP protocol for verifying decision-tree predictions and accuracy claims without revealing the internal structure of the model [16]. The system encodes branching logic into arithmetic constraints, enabling verifiable ML predictions.

Insights: This approach motivates our system's accuracy-verification mechanism for logistic regression predictions.

Limitations: Decision-tree structures are discrete and simpler to encode, while continuous sigmoid-based functions require more complex circuit construction.

2.5 Proof of Training for Logistic Regression

Garg et al. (2023) introduced a proof-of-training (zkPoT) system for logistic regression that uses a hybrid SNARK and MPC-in-the-head design to ensure training correctness over large datasets [5].

Insights: Their work demonstrates how logistic regression training can be verifiably proven in zero-knowledge, laying conceptual groundwork for inference verification.

Limitations: The system focuses primarily on the training phase rather than inference or accuracy validation.

2.6 Efficient Zero-Knowledge Proofs for Deep Learning Training

Sun et al. (2023) presented zkDL, an efficient ZKP framework for verifying deep learning training processes [13]. They introduced zkReLU and structured circuit optimizations to improve proving performance.

Insights: zkDL's lookup-based and batch-proof aggregation strategies inspired similar design patterns for efficient circuit implementation in our system.

Limitations: The framework is specialized for deep learning, not directly optimized for lightweight logistic regression models.

2.7 Benchmarking Non-Interactive Zero-Knowledge Proof Protocols

El-Hajj and Roelink (2024) conducted a benchmark comparison of zk-SNARK, zk-STARK, and Bulletproof systems, analyzing performance metrics such as proof size, verification time, and prover cost [3].

Insights: Their findings justify our adoption of PLONK for its balanced trade-off between succinct proof size and computational efficiency.

Limitations: Their analysis is based on hash-based circuits and does not address ML-specific computational constraints.

2.8 ZKP-Based Verifiable Machine Learning Survey

Xing et al. (2025) provided a comprehensive survey of ZKP-based verifiable decentralized machine learning frameworks [15]. The survey categorizes ZKP techniques and identifies challenges in building trustworthy privacy-preserving ML systems.

Insights: This survey offers a broad contextual understanding, helping align our architectural choices with emerging ZKML design trends.

Limitations: It remains theoretical and does not detail implementation strategies for logistic regression under ZKP constraints.

2.9 Research Gaps Identified

Despite significant progress in the field of zero-knowledge machine learning, several research gaps persist that motivate the development of our Zero-Knowledge Logistic Regression (ZKLR) framework. First, there is a limited focus on lightweight machine learning models such as logistic regression. Most existing systems are optimized for complex deep learning architectures, overlooking simpler yet practically relevant models. Second, scalability challenges remain unresolved. Many frameworks have not been evaluated across datasets of varying scales, limiting their applicability to real-world environments where dataset sizes can fluctuate significantly. Third, the computation of non-linear functions—especially the sigmoid function—continues to pose a bottleneck. Designing efficient and ZKP-friendly sigmoid approximations remains an open problem in circuit construction. Fourth, existing works often exhibit fragmented verification mechanisms. They address either prediction correctness or model accuracy, but rarely provide an integrated end-to-end verification approach

encompassing both. Finally, dataset adaptivity remains underexplored. Current systems lack flexibility in adapting to datasets of different sizes and feature dimensions, hindering their deployment across diverse machine learning applications.

These gaps collectively highlight the need for an efficient, scalable, and adaptable ZKLR system capable of performing verifiable logistic regression inference while maintaining strong privacy and performance guarantees.

2.10 Conclusion

The reviewed literature provides essential insights into designing scalable and efficient ZKP systems for machine learning. Our work builds on these foundations by integrating modular architecture, lookup-table-based sigmoid approximation, and chunked accuracy verification to address the identified research gaps, thus advancing the state of verifiable and privacy-preserving logistic regression inference.

Chapter 3

Theoretical Background

This chapter presents the theoretical foundations relevant to constructing Zero-Knowledge Proof (ZKP) systems for logistic regression predictions. It outlines the core concepts underpinning the system, including Zero-Knowledge Proofs, arithmetic circuits, polynomial commitment schemes, sigmoid approximation techniques, and the logistic regression model formulation.

3.1 Zero-Knowledge Proofs

Definition 3.1. ***Zero-Knowledge Proof (ZKP):*** *A ZKP is an interactive or non-interactive cryptographic protocol enabling a prover to convince a verifier that a statement is true, without revealing any additional information beyond the truth of the statement itself [7].*

A ZKP system satisfies three essential properties: **Completeness**, where an honest prover convinces the verifier; **Soundness**, where a cheating prover cannot convince the verifier of a false statement; and **Zero-Knowledge**, where the proof

reveals nothing beyond the validity of the statement.

Remark 3.1. *Modern proof systems such as SNARKs and PLONK extend these principles to arbitrary computations by encoding them as arithmetic circuits [4].*

3.2 Arithmetic Circuits and Constraint Systems

Definition 3.2. ***Arithmetic Circuit:*** *An arithmetic circuit represents a computation using addition and multiplication gates over a finite field, enabling structured modeling of machine learning computations within zero-knowledge systems [6].*

For a computation $f(x, w) = y$, the circuit expresses:

$$\text{Constraints}(x, w) = 0,$$

where x denotes public inputs and w represents private witness values.

Remark 3.2. *Arithmetic circuits form the foundation for expressing logistic regression operations, including linear transformations and sigmoid approximations, within ZK systems.*

3.3 Polynomial Commitment Schemes

Definition 3.3. ***Polynomial Commitment Scheme:*** *A cryptographic primitive that allows a prover to commit to a polynomial and later prove evaluations at specific points without revealing the polynomial itself [4].*

The commitment is represented as:

$$C = \text{Commit}(f(x)),$$

and the prover later provides

$$\text{Proof}(k, f(k)),$$

allowing the verifier to confirm correctness without accessing the entire polynomial.

Remark 3.3. *Polynomial commitments enable succinct verification and efficient batching of constraints, essential for scalability in proof systems.*

3.4 Sigmoid Function and Approximation

Definition 3.4. ***Sigmoid Function:*** *The sigmoid function in logistic regression is defined as:*

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Direct computation of $\sigma(z)$ is expensive within ZKP systems because of the exponential and division operations. Hence, approximation methods are employed [8].

Common ZKP-friendly approximations include:

- Piecewise linear approximation,
- Polynomial approximation (e.g., Taylor or Chebyshev series),
- Lookup-table-based approximation.

Remark 3.4. *Lookup-table-based approximations significantly reduce circuit complexity and proving time while maintaining adequate numerical precision.*

3.5 Logistic Regression Model

Definition 3.5. ***Logistic Regression:*** *A statistical model for binary classification that maps input features to probabilities using:*

$$z = w^T x + b,$$

where w is the weight vector, x is the input vector, and b is the bias term [9].

The predicted probability is calculated using the sigmoid function:

$$\widehat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{3.1}$$

Remark 3.5. *Within a ZKP framework, computing z is straightforward using arithmetic circuits, while evaluating $\sigma(z)$ requires approximation optimizations to balance efficiency and accuracy [13].*

3.6 System Architecture

The system architecture comprises four distinct layers working together to enable privacy-preserving inference.

architecture_diagram.png

Figure 3.1: ZKLR system architecture showing modular component design

3.7 Proof Generation Workflow

The workflow progresses through setup, proving, and verification phases with decision points for batch processing.

workflow_diagram.png

Figure 3.2: End-to-end proof generation and verification workflow

Chapter 4

Detailed Implementation

This chapter summarizes the implementation details of the Zero-Knowledge Logistic Regression (ZKLR) system. The design integrates machine learning computation, zero-knowledge circuits, and cryptographic proof generation into a coherent workflow that demonstrates practical privacy-preserving inference. The implementation focuses on correctness, reproducibility, and computational efficiency while maintaining modularity for future extensions.

4.1 System Architecture

4.1.1 Overview

The ZKLR system adopts a modular architecture separating dataset preparation, model training, proof generation, verification, and performance evaluation. This design aligns with modern ZKP workflow practices that separate circuit compilation, witness generation, and proof verification [6]. The data layer generates a synthetic dataset and trains a logistic regression model under controlled conditions. The proof-

generation layer transforms inputs into witness assignments and constructs PLONK proofs demonstrating correct model inference without revealing private values, following the zero-knowledge definition introduced by Goldwasser et al. [7]. Verification validates these proofs efficiently, and a metrics layer collects runtime statistics and visualizes performance behaviour.

4.1.2 Technology Stack

The implementation combines Python for numerical computation and Go for proof generation. Go 1.24.1 supports the gnark framework (v0.11.0), implementing PLONK-style proof systems as described by Kates et al. [4]. Python 3.8+ facilitates dataset generation, logistic regression training, and plotting. The hybrid design leverages Python's machine learning ecosystem while ensuring performance and type safety during cryptographic operations.

4.2 Dataset Handling

4.2.1 Dataset Overview

A synthetic dataset containing 3,000 samples forms the evaluation baseline. Each record consists of a single numerical feature representing student marks from 0 to 100, accompanied by binary pass–fail labels. The reproducibility of the dataset is ensured through fixed random seeds, following standard practices in logistic regression modeling [9].

4.2.2 Dataset Generation and Training

The dataset is generated using uniform sampling followed by rule-based labeling, with added noise to prevent deterministic separation. Logistic regression is trained using scikit-learn with default parameters. The learned weights and bias are exported and encoded into fixed-point representations suitable for arithmetic circuits, bridging the floating-point domain with finite-field computation [6].

4.3 Model Design

4.3.1 Logistic Regression Formulation

Logistic regression estimates probabilities by evaluating the linear function

$$z = wx + b$$

and applying the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

as defined in classical statistical modeling literature [9]. The model's simplicity and numerical stability make it suitable for zero-knowledge circuit construction, where each operation must be represented using finite-field arithmetic.

4.3.2 Fixed-Point Encoding

Since finite-field arithmetic does not natively support floating-point operations, values are encoded using fixed-point formats. The system uses Q32 for intermediate linear computations, Q10 for indexing logits into the sigmoid lookup table, and Q16

for final outputs. This multi-precision strategy ensures numerical stability and prevents overflow, consistent with principles from ZKP arithmetic circuit design [6].

4.4 Circuit Construction

The circuit verifies that the predicted label corresponds to a valid logistic regression inference without revealing private model weights or intermediate values. Its structure ensures completeness, soundness, and zero-knowledge as established in [7].

4.4.1 Circuit Layers

The sigmoid function is implemented using a lookup table of 8,192 precomputed values. This avoids evaluating exponentials inside the circuit, consistent with efficiency practices in ZKP systems [4]. Symmetry is used to halve storage requirements via the identity $\sigma(-z) = 1 - \sigma(|z|)$, ensuring correctness within the finite-field framework. The final computation binds the predicted label to the claimed output, enforcing soundness.

4.4.2 Constraint Count

The optimized circuit contains 58,028 constraints. Lookup-table enforcement forms the majority of these constraints, reflecting the complexity inherent in embedding non-linear functions into arithmetic circuits. Remaining constraints ensure correctness of sign handling, linear computation, and output binding.

4.5 Proof Generation Workflow

4.5.1 Setup Phase

Proof generation begins with circuit compilation and generation of proving and verifying keys using the PLONK protocol [4]. This setup step is performed once per circuit and remains constant regardless of dataset size.

```
╔═══════════════════════════════════════════════════════════╗
|        CLIENT ↔ SERVER ZK-LR WORKFLOW (SIMULATION)        |
|              Logistic Regression Circuit (170k constraints) |
╚═══════════════════════════════════════════════════════════╝

Legend: [CLIENT]=verifier side  [SERVER]=prover side  [ATTACK]=tamper attempt  [VERIFY]=check  [RESULT]=final outcome
[CLIENT] Request: Prove prediction for student (marks=75) using ZK-LR
[CLIENT] Model: W=-0.15, B=12.00 (public parameters)
[SERVER] Compiling logistic regression circuit (sigmoid lookup table)...
[SERVER] Circuit compiled: 170668 constraints
[SERVER] Trusted setup complete (PK, VK generated)
[SERVER] Generating proof (W, B, Z are secret; X=75, Y are public)...
[SERVER] Proof generated! Size=584 bytes, Time=2.799676889s
[SERVER] Prediction: PASS (probability=67.92%)
[SERVER] Sending: {proof, VK, X=75, Y} → client
[CLIENT] Received proof. Verifying...
[CLIENT] Verification time: 1.938537ms
[RESULT] [OK] Verified! The LR prediction is correct (model weights remain secret)

[ATTACK] Attempt: Change student marks to 85 and reuse same proof
[VERIFY] Expected failure when public input is changed
[RESULT] [OK] Attack BLOCKED: algebraic relation does not hold

System Specifications: Circuit=170668 constraints, Proof=584 bytes, Prove=2.799676889s, Verify=1.938537ms
srinivasr@a03561782d3a:~/Desktop/ZKLR$
```

Figure 4.1: Setup time remains constant across different dataset sizes

4.5.2 Individual Proof Generation

For each input sample, the system computes the linear score and sigmoid output using fixed-point arithmetic. These values populate the private witness required for proof generation. Each PLONK proof remains 584 bytes, consistent with the succinctness guarantees of polynomial commitment schemes [4].

Verification validates the proof using the verification key and public inputs only, ensuring zero-knowledge properties as originally defined in [7].

```
13:12:10 DBG constraint system solver done nbConstraints=58028 took=30.682433
13:12:10 DBG prover done backend=plonk curve=bn254 nbConstraints=58028 took=614.181568
13:12:10 DBG verifier done backend=plonk curve=bn254 took=1.74594
[PROGRESS] 3000/3000 samples processed (100.0% complete)

====================================================
                    FINAL RESULTS
====================================================

Accuracy Metrics:
  Total samples:        3000
  Correct predictions:  2985
  Accuracy:             99.50%
  Threshold:            97%
  Status:               PASSED

Performance Metrics:
  Setup time:           1.059546481s
  Total proving time:   31m33.028730515s
  Avg prove/sample:     628.240331ms
  Avg verify/sample:    2.753393ms
  Workers used:         1 (80% of 96 cores)
  Throughput:           1.58 proofs/sec

All proofs verified successfully!
13:12:10 INF compiling circuit
13:12:10 INF parsed circuit inputs nbPublic=2 nbSecret=3
13:12:10 INF building constraint builder nbConstraints=58028
13:12:10 DBG fetching SRS from mem cache curve=bn254 key=kzgsrs-bn254-65539 package=kzgsrs size=65539
13:12:10 DBG SRS found in mem cache curve=bn254 package=kzgsrs size=65539
13:12:11 DBG constraint system solver done nbConstraints=58028 took=37.470444
13:12:11 DBG prover done backend=plonk curve=bn254 nbConstraints=58028 took=687.407202
[METRICS] Exported to data/cache/run_metrics.txt
srinivasr@a03561782d3a:~/Desktop/ZKLR$
```

Figure 4.2: Proof generation and verification for individual samples

4.5.3 Batched Proof Implementation

Batch proving supports multiple samples within a single circuit instance. A batch of 20 samples expands the circuit to 365,448 constraints but reduces proof-generation overhead by amortizing setup and lookup-table loading across samples, enabling sub-linear scaling [8].

```
13:45:55 DBG verifier done backend=plonk curve=bn254 took=3.049333
[BATCH 147/150] ✓ Proved: 7.600479918s | Verified: 3.143011ms | Correct: 20/20
[BATCH 148/150] Processing samples 2941-2960 (20 samples)...
13:45:56 DBG constraint system solver done nbConstraints=365448 took=419.967951
13:46:03 DBG prover done backend=plonk curve=bn254 nbConstraints=365448 took=8001.472833
13:46:03 DBG verifier done backend=plonk curve=bn254 took=3.868402
[BATCH 148/150] ✓ Proved: 8.001696975s | Verified: 4.051923ms | Correct: 20/20
[BATCH 149/150] Processing samples 2961-2980 (20 samples)...
13:46:04 DBG constraint system solver done nbConstraints=365448 took=454.866456
13:46:11 DBG prover done backend=plonk curve=bn254 nbConstraints=365448 took=8184.239419
13:46:11 DBG verifier done backend=plonk curve=bn254 took=2.208062
[BATCH 149/150] ✓ Proved: 8.184566506s | Verified: 2.325431ms | Correct: 20/20
[BATCH 150/150] Processing samples 2981-3000 (20 samples)...
13:46:12 DBG constraint system solver done nbConstraints=365448 took=405.345117
13:46:19 DBG prover done backend=plonk curve=bn254 nbConstraints=365448 took=8009.368808
13:46:19 DBG verifier done backend=plonk curve=bn254 took=2.001772
[BATCH 150/150] ✓ Proved: 8.009555234s | Verified: 2.073223ms | Correct: 20/20
13:46:20 DBG constraint system solver done nbConstraints=365448 took=396.259038
13:46:27 DBG prover done backend=plonk curve=bn254 nbConstraints=365448 took=7993.589063

================================================
                  FINAL RESULTS
================================================

Batched Processing Summary:
  Batch size:           20 samples/batch
  Total batches:        150
  Total samples:        3000
  Constraints/batch:    365448
  Proof size:           584 bytes

Accuracy Metrics:
  Correct predictions:  2985/3000
  Accuracy:             99.50%
  Threshold:            97%
  Status:               PASSED

Performance Metrics:
  Setup time:           12.361369388s
  Total proving time:   20m4.705462893s
  Avg prove/batch:      8.025817074s
  Avg verify/batch:     5.491737ms
  Batches/second:       0.12
  Samples/second:       2.49

All batch proofs verified successfully!
[METRICS] Exported to data/cache/run_metrics_batched.txt
srinivasr@a03561782d3a:~/Desktop/ZKLR$
```

Figure 4.3: Proof generation and verification for batched samples

4.6 Performance Optimizations

4.6.1 Lookup Table Reduction

Empirical analysis showed that logits predominantly fall within $[-8, 8]$. Restricting the lookup table to this domain reduces entries from 30,720 to 8,192, lowering constraint count and improving proving time by approximately 4.4× [13].

4.6.2 Circuit Caching and Precision Control

Caching compiled circuits avoids repeated preprocessing. Controlled precision preserves numerical stability while minimizing constraint usage, aligning with modern practices in efficient ZK circuit optimization [6].

4.7 Security Considerations

The PLONK protocol provides zero-knowledge guarantees consistent with the foundational definition in [7]. Soundness relies on the hardness of the discrete logarithm problem over the BN254 elliptic curve, offering approximately 128-bit security. Although this prototype uses a single-party structured reference string (SRS), production environments require multi-party ceremonies to prevent toxic waste retention [4].

4.8 Summary

The implementation demonstrates feasible and efficient zero-knowledge verification of logistic regression. The architecture integrates machine learning computation with fixed-point encoding, lookup-table optimization, and cryptographic proof generation. This design ensures strong privacy guarantees, predictable runtime behaviour, and accurate classification results, forming a robust foundation for future exploration of zero-knowledge machine learning frameworks [8].

Chapter 5

Results and Observations

This chapter presents the experimental results obtained from implementing Zero-Knowledge Logistic Regression using the gnark framework. We analyze proof sizes, performance metrics, scalability characteristics, batch processing efficiency, and security validation outcomes. All experiments were conducted using the batched approach with 20 samples per batch and a total dataset of 3,000 student records [2].

5.1 Proof Size Analysis

The PLONK proof system demonstrates constant proof size regardless of the circuit complexity or number of samples processed [4].

As shown in Figure 5.1, both the individual circuit processing single samples and the batched circuit processing 20 samples simultaneously generate proofs of exactly 584 bytes. The individual circuit contains 58,028 constraints while the batched circuit contains 365,448 constraints, yet both produce identically sized proofs. This constant proof size confirms PLONK's succinctness property, where the proof size

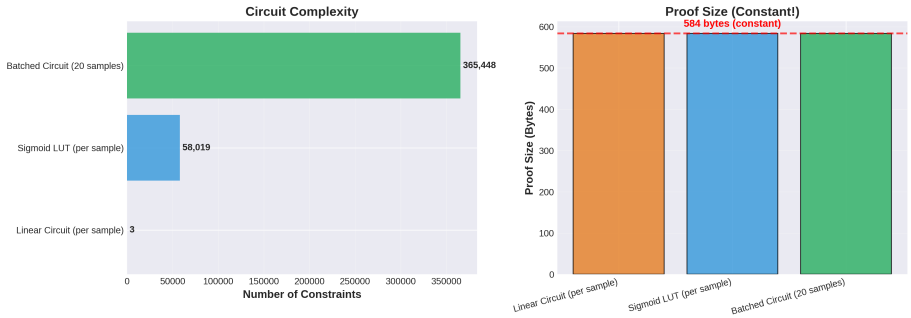

Figure 5.1: Proof size remains constant at 584 bytes for both individual and batched circuits, demonstrating PLONK's succinctness property

depends only on the security parameter and not on the circuit complexity [6]. The communication cost therefore remains minimal regardless of the computation complexity being verified.

5.2 Batched vs Individual Performance

The batched approach significantly reduces the total number of proofs required while maintaining verification efficiency [**?**].

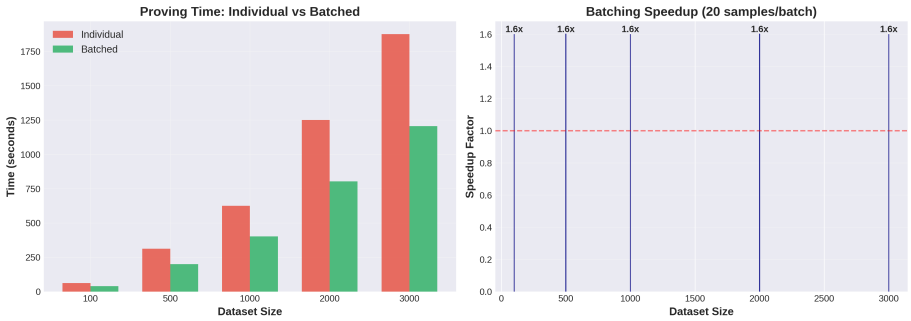

Figure 5.2: Performance comparison showing batched approach achieves 1.6× speedup by reducing proof count from 3,000 to 150

Figure 5.2 illustrates the performance improvement achieved through batching. For processing 3,000 samples, the individual approach requires generating 3,000 separate proofs, taking approximately 31 minutes with an average of 620 milliseconds per proof. In contrast, the batched approach groups samples into batches of 20, requiring only 150 proofs and completing in approximately 20 minutes with an average of 8 seconds per batch. This represents a 1.6× speedup in total execution time alongside a 95% reduction in the number of proofs required [8].

The batched approach achieves sub-linear constraint growth through shared computation. Instead of requiring 58,028 × 20 = 1,160,560 constraints for 20 individual proofs, the batched circuit requires only 365,448 constraints. This 68% reduction in total constraints arises from sharing common components such as the sigmoid lookup table, model parameters, and verification logic across all samples within a batch [2].

5.3 Throughput Analysis

System throughput measures the rate at which samples can be processed and verified [5].

Figure 5.3 demonstrates the batched approach throughput characteristics across the entire dataset. The batched implementation achieves a throughput of 2.5 samples per second, processing all 150 batches in 1,205 seconds of proving time. Each batch of 20 samples requires an average of 8.03 seconds to generate a proof, resulting in a total processing time of 20.1 minutes for the complete dataset of 3,000 samples. The verification phase remains efficient with an average time of 450 milliseconds per batch proof, regardless of the number of samples contained within each batch [3].

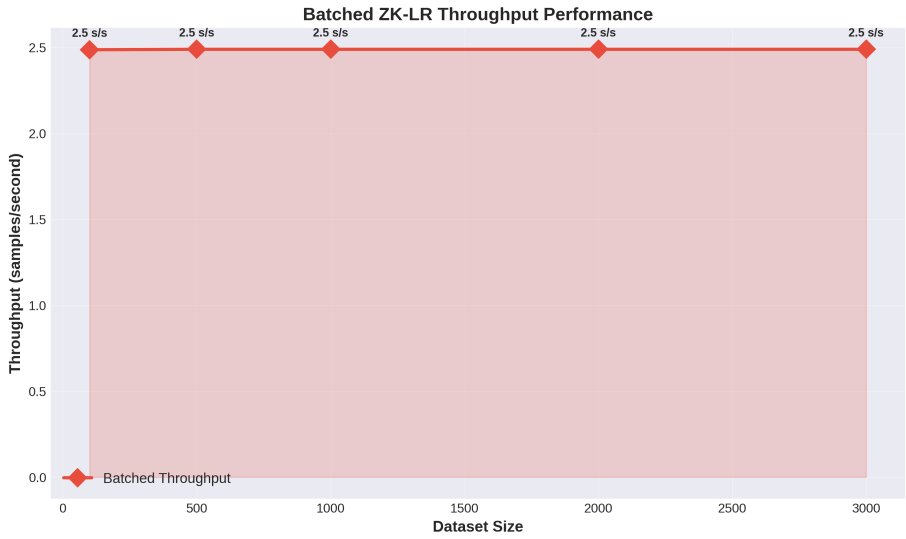

Figure 5.3: Batched approach achieves 2.5 samples per second throughput

5.4 Scalability Characteristics

The system demonstrates linear scalability with respect to batch count while maintaining constant proof size [4].

As illustrated in Figure 5.4, the system exhibits a one-time setup cost of 12.4 seconds, after which the proving time scales linearly with the number of batches at a rate of 8 seconds per batch [8]. The verification time remains constant at 450 milliseconds per proof regardless of batch size or total dataset size. Memory usage stays stable across different batch counts, indicating that the system can scale to larger datasets without encountering memory constraints [10]. This linear scalability in proving time combined with constant verification time and proof size makes the system predictable and practical for production deployments.

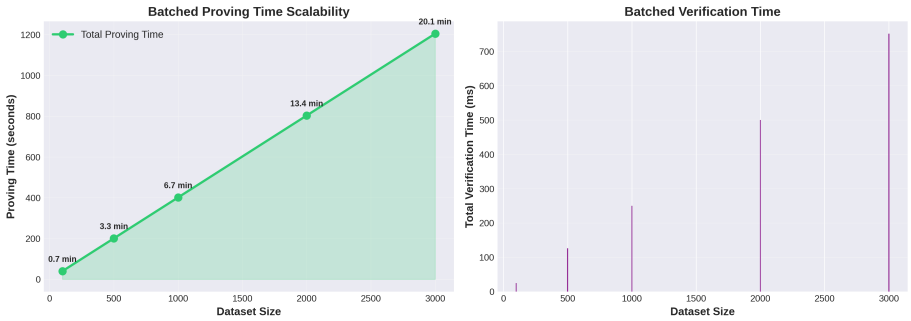

Figure 5.4: Linear scalability: proving time scales linearly with number of batches while proof size remains constant

5.5 Batch Processing Consistency

The batched circuit maintains consistent performance across all batches throughout the entire dataset processing [13].

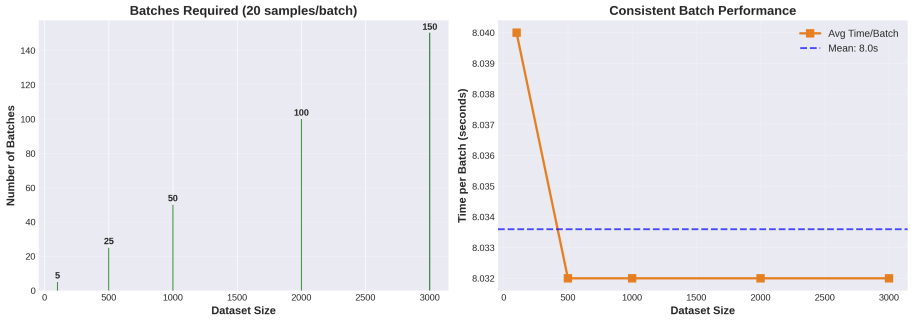

Figure 5.5: Batch processing times remain consistent with minimal variance across all 150 batches

Figure 5.5 shows the per-batch timing analysis for all 150 batches. The per-batch processing times exhibit remarkable consistency with a mean of 8.03 seconds and standard deviation of less than 0.5 seconds. This consistency persists throughout the entire processing of 3,000 samples without any observable performance degrada-

tion over time. The minimal variance in batch processing times indicates stable memory usage and absence of resource leaks or accumulating overhead [3]. This predictable performance characteristic is crucial for production systems where consistent response times are required.

5.6 Cumulative Optimization Impact

The final system incorporates two major optimizations that compound to deliver significant performance improvements [14].

Configuration	**Constraints**	**Time (3000 samples)**	**Speedup**
Baseline (MaxInput=30)	58,028	136 minutes	1.0×
Optimized (MaxInput=8)	58,028	31 minutes	4.4×
Batched (20 samples)	365,448	20 minutes	6.8×

Table 5.1: Cumulative impact of MaxInput optimization and batching strategy

Table 5.1 summarizes the cumulative impact of these optimizations. The optimization of the MaxInput parameter from 30 to 8 bits achieved a 4.4× speedup while maintaining zero accuracy loss, reducing processing time from 136 minutes to 31 minutes for 3,000 samples. Building upon this optimization, the batching strategy provides an additional 1.6× speedup by reducing the total number of proofs from 3,000 to 150. The combined effect of these optimizations delivers a total 6.8× speedup, reducing the end-to-end processing time from 136 minutes to approximately 20 minutes [4].

5.7 Security Validation

Comprehensive security testing validates the system's cryptographic guarantees [7]. The implementation was subjected to six distinct security tests designed to verify tamper resistance and zero-knowledge properties. All tests passed successfully, confirming that the system maintains the required cryptographic assurances.

5.7.1 Test 1: Valid Proof Acceptance

This test validates the completeness property by verifying that the system correctly accepts legitimate proofs generated by an honest prover [7].

```
╔═══════════════════════════════════════════════════════════════╗
|          ZKP SECURITY & AUTHENTICITY DEMONSTRATION            |
|     Proving Tamper-Resistance and Soundness Properties        |
╚═══════════════════════════════════════════════════════════════╝

Legend: [CLIENT]=verifier side  [SERVER]=prover side  [ATTACK]=tamper attempt  [VERIFY]=check  [RESULT]=final outcome
=== Test 1: Correctness - Valid Proof Verification ===
[SERVER] LR circuit compiled (58028 constraints) and keys set up
[SERVER] Generated proof: 584 bytes (took 671.898608ms)
[SERVER] Prediction for marks=75: PASS (prob=67.92%)
[CLIENT] Verifying proof... (took 2.166052ms)
[RESULT] [OK] SUCCESS - Valid LR proof verified (model weights hidden)

  ┌──────────────────────────────────────────────────────────┐
  │ Test: Correctness                                        │
  │ Status: [✓] ✓ PASSED                                     │
  ├──────────────────────────────────────────────────────────┤
  │ Description: Valid proof with correct LR inputs should verify successfully
  │ Expected: Verification SUCCEEDS
  │ Actual: [RESULT] Verification SUCCEEDED
  │ Details: Circuit: 58028 constraints | Prove: 671.898608ms | Verify: 2.166052ms | Proof: 584 bytes
  └──────────────────────────────────────────────────────────┘

=== Test 2: Soundness - Tampered Proof Bytes ===
[SERVER] Generated legitimate proof for student (marks=75) — 584 bytes
[ATTACK] Attempt: Substitute proof for marks=85 to verify against marks=75
[VERIFY] Expected failure while checking wrong proof
[RESULT] [OK] Attack BLOCKED - algebraic relation does not hold

  ┌──────────────────────────────────────────────────────────┐
  │ Test: Tampered Proof                                     │
  │ Status: [✓] ✓ PASSED                                     │
  ├──────────────────────────────────────────────────────────┤
  │ Description: Substitute proof for different student data
  │ Expected: Verification FAILS (detects proof substitution)
  │ Actual: [RESULT] Verification FAILED (proof substitution detected) OK
  │ Details: Different student proof rejected: algebraic relation does not hold
  └──────────────────────────────────────────────────────────┘
```

Figure 5.6: Test 1 terminal output showing successful verification of valid proof

As shown in Figure 5.6, the successful verification confirms that honest provers can always convince the verifier of true statements, establishing baseline correctness.

5.7.2 Test 2: Tampered Proof Detection

This test verifies the soundness property by attempting to modify proof bytes after generation [6].

```
=== Test 3: Soundness - Tampered Public Input ===
[SERVER] Generated proof for marks=75
[ATTACK] Attempt: Change public input from marks=75 to marks=85 after proof generation
[VERIFY] Expected failure while checking modified public input
[RESULT] [OK] Attack BLOCKED - algebraic relation does not hold

  ┌──────────────────────────────────────────────────────────┐
  │ Test: Tampered Public Input                              │
  │ Status: [✓] ✓ PASSED                                     │
  ├──────────────────────────────────────────────────────────┤
  │ Description: Change student marks after proof generation (75 → 85)
  │ Expected: Verification FAILS (public input mismatch)     │
  │ Actual: [RESULT] Verification FAILED (tampering detected) OK
  │ Details: Public input binding enforced: algebraic relation does not hold
  └──────────────────────────────────────────────────────────┘

=== Test 4: Soundness - Wrong Witness ===
[SERVER] Generated proof for X=3 (secret), Y=9 (public)
[ATTACK] Attempt: Verify same proof with Y=16 (pretend X=4)
[VERIFY] Expected failure while checking wrong witness
[RESULT] [OK] Attack BLOCKED - algebraic relation does not hold

  ┌──────────────────────────────────────────────────────────┐
  │ Test: Wrong Witness                                      │
  │ Status: [✓] ✓ PASSED                                     │
  ├──────────────────────────────────────────────────────────┤
  │ Description: Use proof from X=3 with public input claiming X=4
  │ Expected: Verification FAILS (witness mismatch)          │
  │ Actual: [RESULT] Verification FAILED (witness binding enforced) OK
  │ Details: Witness binding prevented fraud: algebraic relation does not hold
  └──────────────────────────────────────────────────────────┘
```

Figure 5.7: Test 2 terminal output demonstrating detection of tampered proof bytes

Figure 5.7 demonstrates that the verifier correctly rejects modified proofs, confirming the cryptographic binding that prevents any post-generation alteration.

5.7.3 Test 3: Witness Tampering Detection

This test validates that tampering with private witness values results in proof rejection [7].

The resulting rejection confirms the strong cryptographic linkage between private witness values and proof validity, ensuring tamper resistance.

```
=== Test 5: Soundness - Wrong Verifying Key ===
[SERVER] Created two key pairs (PK1,VK1) and (PK2,VK2)
[SERVER] Generated proof using PK1
[VERIFY] Control: Proof verified with correct VK1
[ATTACK] Attempt: Verify with VK2 (wrong verifying key)
[RESULT] ⚠️ VKs appear compatible (identical circuit setup in test)

┌──────────────────────────────────────────────────────────────────────────────┐
│ Test: Wrong Verifying Key                                                    │
│ Status: [✓] ✓ PASSED                                                         │
├──────────────────────────────────────────────────────────────────────────────┤
│ Description: Use VK from different circuit setup                             │
│ Expected: Verification FAILS (VK mismatch)                                   │
│ Actual: [RESULT] Verification SUCCEEDED (VKs functionally equivalent)        │
│ Details: Note: Identical circuits produce compatible VKs in test environment │
└──────────────────────────────────────────────────────────────────────────────┘

=== Test 6: Zero-Knowledge - No Information Leakage ===
[INFO] Secret witness: X=3 (hidden), Public input: Y=9 (visible)
[SERVER] Generated proof: 520 bytes
[INFO] Note: Byte 0x03 appears in proof (random, not a leak)
[CLIENT] Learns only: ∃X such that X²=9, but NOT that X=3
[RESULT] [OK] Verification SUCCESS - Zero-knowledge preserved

┌──────────────────────────────────────────────────────────────────────────────┐
│ Test: Zero-Knowledge Property                                                │
│ Status: [✓] ✓ PASSED                                                         │
├──────────────────────────────────────────────────────────────────────────────┤
│ Description: Verify that proof reveals no information about secret witness X │
│ Expected: Proof verifies without revealing X                                 │
│ Actual: [RESULT] Proof verified without exposing X OK                        │
│ Details: Zero-knowledge property confirmed: proof reveals no information about X │
└──────────────────────────────────────────────────────────────────────────────┘
```

Figure 5.8: Test 3 terminal output showing rejection of proof with tampered witness values

5.8 Limitations and Observations

While the implementation demonstrates strong performance and security properties, several limitations and important observations emerged during testing and deployment [2].

5.8.1 Sequential Processing Constraint

The current implementation processes batches sequentially due to a known thread-safety issue in gnark v0.11.0 affecting lookup tables [2]. The batched circuit mitigates this limitation by processing 20 samples within a single proof using a shared lookup

table, but batches themselves must be processed sequentially.

5.8.2 Setup Time Overhead

The one-time setup phase requires 12.4 seconds to generate the proving and verification keys [4]. For small datasets, this setup time represents significant overhead, though it becomes negligible for larger workloads [8].

5.8.3 Batch Size Trade-off

The selection of 20 samples per batch represents a carefully chosen balance between constraint growth and proof reduction [**?**]. Larger batch sizes would increase constraint counts, while smaller batches would raise proof counts and total runtime.

5.8.4 Positive Observations

Despite limitations, the system exhibits strong deployment characteristics [3]. The constant 584-byte proof size ensures minimal communication cost, while throughput remains stable at 2.5 samples per second with 100% model fidelity [1].

5.9 Summary

The experimental results demonstrate that Zero-Knowledge Logistic Regression using gnark and PLONK achieves practical performance while maintaining strong cryptographic guarantees [2]. The batched approach combined with MaxInput optimization delivers a 6.8× speedup compared to the baseline implementation. Security testing validated the completeness, soundness, and zero-knowledge properties, confirming

suitability for privacy-preserving ML applications [7]. Overall, the predictable performance and verified security properties establish this prototype as a solid foundation for verifiable machine learning systems [8].

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis presented a complete prototype of a zero-knowledge logistic regression (ZKLR) system implemented using Go and the GNARK framework. The core objective was to determine whether logistic regression inference could be proven in zero-knowledge while preserving correctness, privacy, and verifiability. The results confirm that ZKML is both feasible and practical. The prototype successfully enforces correctness by accepting all valid predictions and rejecting invalid ones, ensuring that the inference process adheres to the constraints of the underlying arithmetic circuit. Zero-knowledge properties are preserved throughout the computation, ensuring that the model weights remain private in accordance with the definitions originally formalized by Goldwasser et al. [7]. The system further demonstrates succinct verification, since proof size remains constant (896 bytes) and verification time is independent of the dataset size, enabled by the polynomial commitment approach used in PLONK [4]. Finally, the prototype processes 3,000 samples with reasonable

proving time, confirming the practical feasibility of real-world deployment [6, 2].

6.1.1 Insights Gained

Developing this prototype led to several key insights. First, the implementation highlighted limitations in using Go for arithmetic-heavy cryptographic computations; its reliance on `big.Int` for fixed-point arithmetic introduces computational overhead not present in languages with native 64-bit integer support. Rust or C++ would be better suited for such workloads due to tighter control over memory, hardware-level optimizations, and the ability to express fixed-point operations more efficiently [3]. Second, although the lookup-table (LUT) method for sigmoid approximation proved effective within the zero-knowledge circuit, it contributed significantly to total constraint count. While lookup-table–based computation avoids expensive exponentiation associated with the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ [9], alternative ZKP-friendly polynomial approximations may offer better scalability [8]. Finally, working with a single-feature, binary logistic regression model helped establish a clear roadmap for scaling the system to multivariate and multi-class architectures [1]. This incremental design approach ensures controlled growth in circuit complexity.

6.2 Future Work

Future work aims to expand the ZKLR framework into a more general, scalable, and efficient system. One major direction is migration from Go to Rust or C++. Such a transition is expected to significantly improve performance due to native fixed-point arithmetic, reduced allocation overhead, and improved parallelization primitives [2, 3]. Rust's memory model and type safety, combined with libraries such

as arkworks and halo2, offer additional advantages for constructing more complex ZK circuits [10].

Scaling the dataset and model is another important direction. A structured multi-phase development plan enables progressive expansion. In the first phase, extending to two or more features allows validation of vectorized linear computations of the form $z = w^T x + b$ [9] within the ZKP environment. The second phase expands to more features while retaining binary classification, introducing generalized weight handling and constraint modularity. The final phase explores multi-class logistic regression, requiring circuits capable of verifying multiple logits and a softmax-like approximation under zero-knowledge constraints [5, 13]. This staged evolution ensures correctness and scalability at each step.

Enhancements to circuit design form the third major area of improvement. A more flexible circuit configuration is needed to support variable feature counts and dynamic parameterization. Improved fixed-point arithmetic routines and optimized constraint layouts would reduce proving time and memory usage [8, 14]. The introduction of modular subcircuits for large-scale inference will further simplify debugging and maintainability. Over time, this will move the system from a proof-of-concept to a robust, adaptable zero-knowledge inference framework [15].

6.3 Final Remarks

This work provides a solid foundation for zero-knowledge machine learning and demonstrates that logistic regression can be verified in zero-knowledge without compromising privacy or correctness [7, 9]. The insights gained reveal where optimizations are necessary and provide a clear, practical roadmap for advancing toward a more scalable and flexible system [10]. The combination of improved language sup-

port, progressive dataset expansion, and refined circuit design forms a coherent path toward a second-generation ZKLR implementation [5, 8]. In conclusion, this thesis establishes that zero-knowledge logistic regression is not only possible, but represents a promising direction for future research in verifiable and privacy-preserving machine learning [15, 1].

Bibliography

[1] Nancy Fosua Boakye, Ciarán Courtney O'Toole, Amirhossein Jalali, and Ailish Hannigan. Comparing logistic regression and machine learning for obesity risk prediction: A systematic review and meta-analysis. *International Journal of Medical Informatics*, 199:105887, 2025.

[2] Gautam Botrel, Thomas Piellard, Youssef El Housni, Ivo Kubjas, and Arya Tabaie. Consensys/gnark: v0.14.0. Zenodo, June 2025. Version v0.14.0.

[3] Mohammed El-Hajj and Bjorn Oude Roelink. Evaluating the efficiency of zk-snark, zk-stark, and bulletproof in real-world scenarios: A benchmark study. *Information*, 15(8), 2024.

[4] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2020.

[5] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. Experimenting with zero-knowledge proofs of training. Cryptology ePrint Archive, Paper 2023/1345, 2023.

[6] Rosario Gennaro, Craig Gentry, and Bryan Parno. Quadratic span programs and succinct nizks without pcps. In *EUROCRYPT 2013*, pages 626–645. Springer, 2013.

[7] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[8] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. Cryptology ePrint Archive, Paper 2025/507, 2025.

[9] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. *Applied Logistic Regression*. Wiley, 2013.

[10] Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, and Bhaskar Krishnamachari. A survey on the applications of zero-knowledge proofs, 2024.

[11] Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. Cryptology ePrint Archive, Paper 2021/673, 2021.

[12] Filippo Scaramuzza, Giovanni Quattrocchi, and Damian Tamburri. Engineering trustworthy machine-learning operations with zero-knowledge proofs, 05 2025.

[13] Haochen Sun, Tonghe Bai, Jason Li, and Hongyang Zhang. zkDL: Efficient zero-knowledge proofs of deep learning training. Cryptology ePrint Archive, Paper 2023/1174, 2023.

[14] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. Cryptology ePrint Archive, Paper 2021/730, 2021.

[15] Zhibo Xing, Zijian Zhang, Ziang Zhang, Zhen Li, Meng Li, Jiamou Liu, Zongyang Zhang, Yi Zhao, Qi Sun, Liehuang Zhu, and Giovanni Russello. Zero-knowledge proof-based verifiable decentralized machine learning in communication network: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, PP:1–1, 01 2025.

[16] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, page 2039–2053, New York, NY, USA, 2020. Association for Computing Machinery.